UNIT-II

Introduction to Linear Data Structures: Definition and importance of linear data structures, Abstract data types (ADTs) and their implementation, Overview of time and space complexity analysis for linear data structures.
Sorting Techniques: Quick sort, Merge sort, Radix sort

Linear Data Structures:

Linear Data Structures are a type of data structure in computer science where data elements are arranged sequentially or linearly. Each element has a previous and next adjacent, except for the first and last elements.

Characteristics of Linear Data Structure:(Importance)

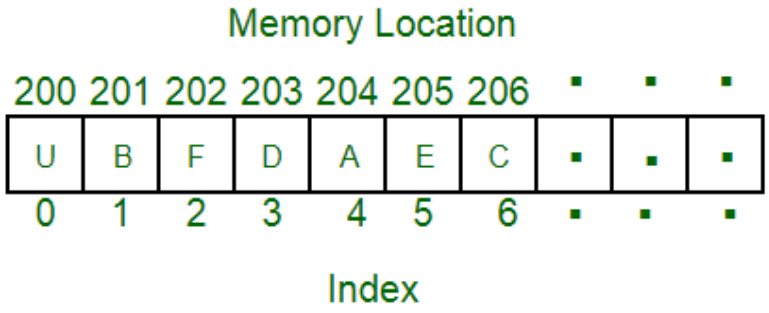
- **Sequential Organization:** In linear data structures, data elements are arranged sequentially, one after the other. Each element has a unique predecessor (except for the first element) and a unique successor (except for the last element)
- **Order Preservation:** The order in which elements are added to the data structure is preserved. This means that the first element added will be the first one to be accessed or removed, and the last element added will be the last one to be accessed or removed.
- **Fixed or Dynamic Size**: Linear data structures can have either fixed or dynamic sizes. Arrays typically have a fixed size when they are created, while other structures like linked lists, stacks, and queues can dynamically grow or shrink as elements are added or removed.
- **Efficient Access:** Accessing elements within a linear data structure is typically efficient. For example, arrays offer constant-time access to elements using their index.

Linear data structures are commonly used for organising and manipulating data in a sequential fashion. Some of the most common linear data structures include:

- **Arrays:** A collection of elements stored in contiguous memory locations.
- **Linked Lists:** A collection of nodes, each containing an element and a reference to the next node.
- **Stacks:** A collection of elements with **Last-In-First-Out (LIFO)** order.
- **Queues:** A collection of elements with **First-In-First-Out (FIFO)** order.

Array

An array is a collection of items of same data type stored at contiguous memory locations.

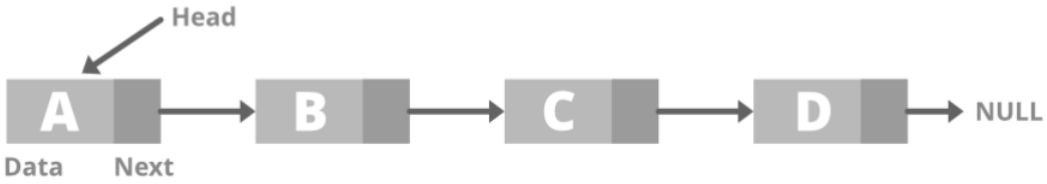


Linked List

A L**inked List** is a linear data structure which looks like a chain of nodes, where each node contains a **data** field and a **reference(link)** to the next node in the list. Unlike Arrays, Linked List elements are not stored at a contiguous location.

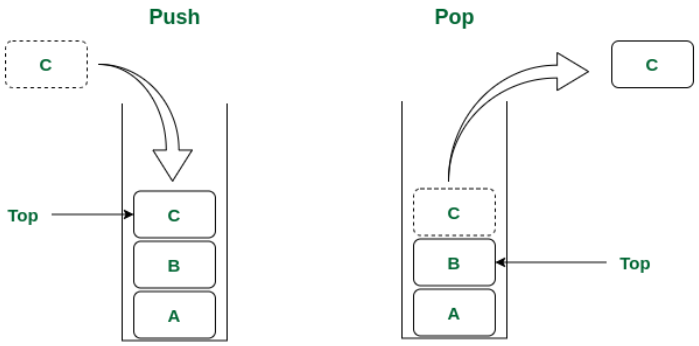
Common Features of Linked List:

- **Node:** Each element in a linked list is represented by a node, which contains two components:
    - **Data:** The actual data or value associated with the element.
    - **Next Pointer(or Link):** A reference or pointer to the next node in the linked list.
- **Head:** The first node in a linked list is called the "head." It serves as the starting point for traversing the list.
- **Tail:** The last node in a linked list is called the "tail."



Stack:

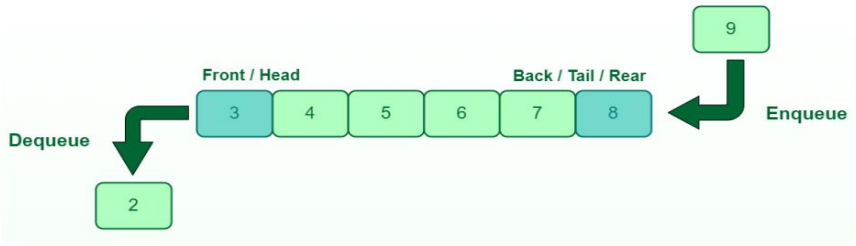
A stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle, meaning that the last element added to the stack is the first one to be removed.



Stack Data Structure

Queue:

A queue is a linear data structure that follows the First-In-First-Out (FIFO) principle. In a queue, the first element added is the first one to be removed.



Queue Data Structure

Advantages of Linear Data Structures

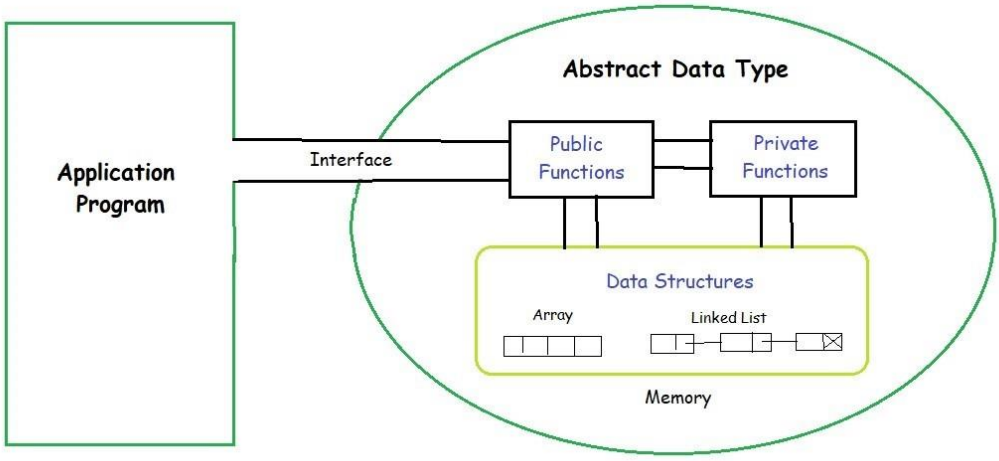
- **Efficient data access:** Elements can be easily accessed by their position in the sequence.
- **Dynamic sizing:** Linear data structures can dynamically adjust their size as elements are added or removed.
- **Ease of implementation:** Linear data structures can be easily implemented using arrays or linked lists.
- **Versatility**: Linear data structures can be used in various applications, such as searching, sorting, and manipulation of data.
- **Simple algorithms**: Many algorithms used in linear data structures are simple and straightforward.

Disadvantages of Linear Data Structures

- **Limited data access**: Accessing elements not stored at the end or the beginning of the sequence can be time-consuming.
- **Memory overhead**: Maintaining the links between elements in linked lists and pointers in stacks and queues can consume additional memory.
- **Complex algorithms**: Some algorithms used in linear data structures, such as searching and sorting, can be complex and time-consuming.
- **Inefficient use of memory**: Linear data structures can result in inefficient use of memory if there are gaps in the memory allocation.
- **Unsuitable for certain operations**: Linear data structures may not be suitable for operations that require constant random access to elements, such as searching for an element in a large dataset.

Abstract Data Types:

- An Abstract Data Type (ADT) is a conceptual model that defines a set of operations and behaviors for a data structure, without specifying how these operations are implemented or how data is organized in memory.
- The definition of ADT only mentions what operations are to be performed but not how these operations will be implemented. It does not specify how data will be organized in memory and what algorithms will be used for implementing the operations.
- It is called “abstract” because it provides an implementation-independent view.

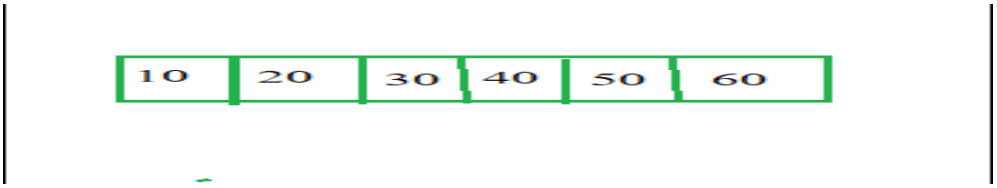


For example, we use primitive values like int, float, and char with the understanding that these data types can operate and be performed on without any knowledge of their implementation details. ADTs operate similarly by defining what operations are possible without detailing their implementation.

Types of ADT`s:

List ADT:

The List ADT (Abstract Data Type) is a sequential collection of elements that supports a set of operations without specifying the internal implementation. It provides an ordered way to store, access, and modify data.

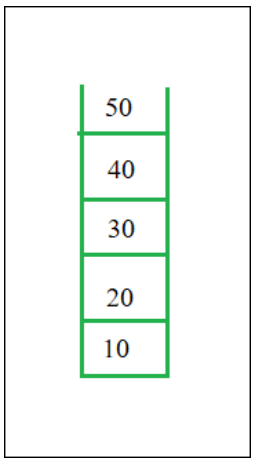


The List ADT need to store the required data in the sequence and should have the following operations**:**

- **get():** Return an element from the list at any given position.
- **insert():** Insert an element at any position in the list.
- **remove():** Remove the first occurrence of any element from a non-empty list.
- **removeAt():** Remove the element at a specified location from a non-empty list.
- **replace():** Replace an element at any position with another element.
- **size():** Return the number of elements in the list.
- **isEmpty():** Return true if the list is empty; otherwise, return false.
- **isFull():** Return true if the list is full; otherwise, return false.

Stack ADT:

The Stack ADT is a linear data structure that follows the LIFO (Last In, First Out) principle. It allows elements to be added and removed only from one end, called the top of the stack.

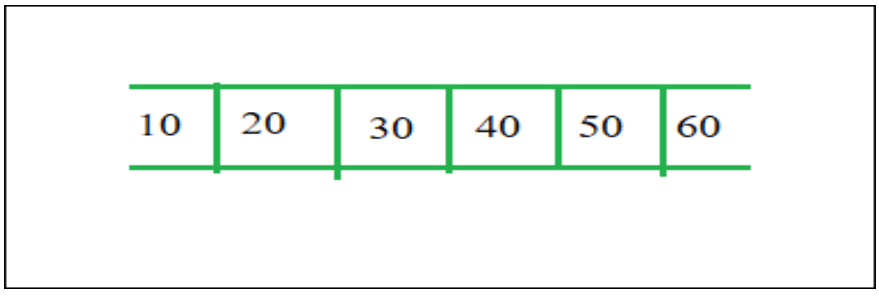


In Stack ADT, the order of insertion and deletion should be according to the FILO or LIFO Principle. Elements are inserted and removed from the same end, called the top of the stack. It should also support the following operations:

- **push():** Insert an element at one end of the stack called the top.
- **pop():** Remove and return the element at the top of the stack, if it is not empty.
- **peek():** Return the element at the top of the stack without removing it, if the stack is not empty.
- **size():** Return the number of elements in the stack.
- **isEmpty():** Return true if the stack is empty; otherwise, return false.
- **isFull():** Return true if the stack is full; otherwise, return false.

Queue ADT:

The Queue ADT is a linear data structure that follows the FIFO (First In, First Out) principle. It allows elements to be inserted at one end (rear) and removed from the other end (front).



The Queue ADT follows a design similar to the Stack ADT, but the order of insertion and deletion changes to FIFO. Elements are inserted at one end (called the rear) and removed from the other end (called the front). It should support the following operations:

- **enqueue():** Insert an element at the end of the queue.
- **dequeue():** Remove and return the first element of the queue, if the queue is not empty.
- **peek():** Return the element of the queue without removing it, if the queue is not empty.
- **size():** Return the number of elements in the queue.
- **isEmpty():** Return true if the queue is empty; otherwise, return false.

Advantage of ADTs:

- **Encapsulation**: ADTs provide a way to encapsulate data and operations into a single unit, making it easier to manage and modify the data structure.
- **Abstraction**: ADTs allow users to work with data structures without having to know the implementation details, which can simplify programming and reduce errors.
- **Data Structure Independence**: ADTs can be implemented using different data structures, which can make it easier to adapt to changing needs and requirements.
- **Information Hiding**: ADTs can protect the integrity of data by controlling access and preventing unauthorized modifications.
- **Modularity**: ADTs can be combined with other ADTs to form more complex data structures, which can increase flexibility and modularity in programming.

Disadvantages of ADTs:

- **Overhead**: Implementing ADTs can add overhead in terms of memory and processing, which can affect performance.
- **Complexity**: ADTs can be complex to implement, especially for large and complex data structures.

- **Learning** Curve: Using ADTs requires knowledge of their implementation and usage, which can take time and effort to learn.
- **Limited Flexibility:** Some ADTs may be limited in their functionality or may not be suitable for all types of data structures.
- **Cost**: Implementing ADTs may require additional resources and investment, which can increase the cost of development.

Time Complexity and Space Complexity:

- The **time complexity** of an algorithm quantifies the amount of time taken by an algorithm to run as a function of the length of the input. Note that the time to run is a function of the length of the input and not the actual execution time of the machine on which the algorithm is running on.
- The **space complexity** of an algorithm quantifies the amount of space taken by an algorithm to run as a function of the length of the input.

Some general time complexities are listed below with the input range for which they are accepted in competitive programming:

Input Length	**Worst Accepted Time Complexity**	**Usually type of solutions**
10 -12	O(N!)	Recursion and backtracking
15-18	O(2N * N)	Recursion, backtracking, and bit manipulation
18-22	O(2N * N)	Recursion, backtracking, and bit manipulation
30-40	O(2N/2 * N)	Meet in the middle, Divide and Conquer
100	O(N4)	Dynamic programming, Constructive

400	O(N3)	Dynamic programming, Constructive
2K	O(N2* log N)	Dynamic programming, Binary Search, Sorting, Divide and Conquer
10K	O(N2)	Dynamic programming, Graph, Trees, Constructive
1M	O(N* log N)	Sorting, Binary Search, Divide and Conquer
100M	O(N), O(log N), O(1)	Constructive, Mathematical, Greedy Algorithms

Asymptotic Notations:

- Asymptotic Notations are mathematical tools used to analyze the performance of algorithms by understanding how their efficiency changes as the input size grows.
- These notations provide a concise way to express the behavior of an algorithm's time or space complexity as the input size approaches infinity.
- Rather than comparing algorithms directly, asymptotic analysis focuses on understanding the relative growth rates of algorithms' complexities.
- It enables comparisons of algorithms' efficiency by abstracting away machine-specific constants and implementation details, focusing instead on fundamental trends.
- Asymptotic analysis allows for the comparison of algorithms' space and time complexities by examining their performance characteristics as the input size varies.

- By using asymptotic notations, such as Big O, Big Omega, and Big Theta, we can categorize algorithms based on their worst-case, best-case, or average-case time or space complexities, providing valuable insights into their efficiency.

There are mainly three asymptotic notations:

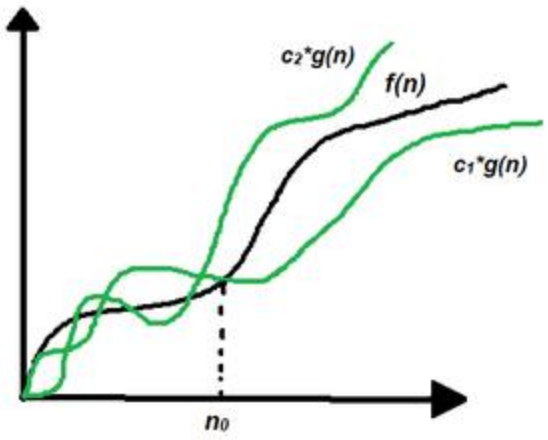
1. Big-O Notation (O-notation)
2. Omega Notation (Ω-notation)
3. Theta Notation (Θ-notation)

Theta Notation (Θ-Notation):

Theta notation encloses the function from above and below. Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the average-case complexity of an algorithm.

Theta (Average Case) You add the running times for each possible input combination and take the average in the average case.

Let g and f be the function from the set of natural numbers to itself. The function f is said to be Θ(g), if there are constants c1, c2 > 0 and a natural number n0 such that c1* g(n) ≤ f(n) ≤ c2 * g(n) for all n ≥ n0



Big-O Notation (O-notation):

Big-O notation represents the upper bound of the running time of an algorithm. Therefore, it gives the worst-case complexity of an algorithm.

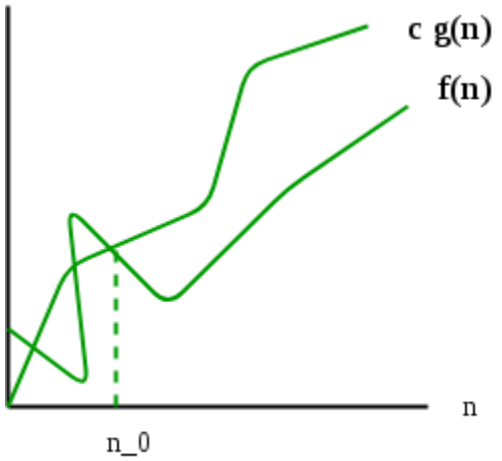
.It is the most widely used notation for Asymptotic analysis.
.It specifies the upper bound of a function.

.The maximum time required by an algorithm or the worst-case time complexity.
.It returns the highest possible output value(big-O) for a given input.
.Big-O(Worst Case) It is defined as the condition that allows an algorithm to complete statement execution in the longest amount of time possible.

If f(n) describes the running time of an algorithm, f(n) is O(g(n)) if there exist a positive constant C and n0 such that, $0 \leq f(n) \leq cg(n)$ for all $n \geq n0$

It returns the highest possible output value (big-O)for a given input.

The execution time serves as an upper bound on the algorithm's time complexity.



f(n) = O(g(n))

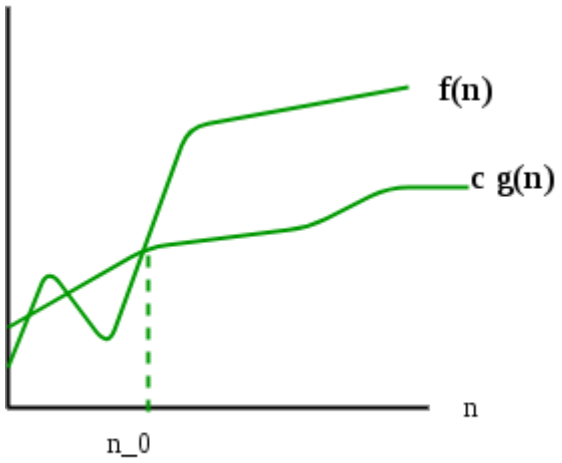
<u>Omega Notation (Ω-Notation)</u>:

Omega notation represents the lower bound of the running time of an algorithm. Thus, it provides the best case complexity of an algorithm.

The execution time serves as a lower bound on the algorithm's time complexity.

It is defined as the condition that allows an algorithm to complete statement execution in the shortest amount of time.

Let g and f be the function from the set of natural numbers to itself. The function f is said to be Ω(g), if there is a constant $c > 0$ and a natural number n0 such that $c*g(n) \leq f(n)$ for all $n \geq n0$



f(n) = Omega(g(n))

Sorting Techniques:

A Sorting Algorithm is used to rearrange a given array or list of elements in an order. Sorting is provided in library implementation of most of the programming languages.

Quick Sort Algorithm:

Quick Sort is one of the best sorting algorithms that follows the **divide-and-conquer** approach like Merge Sort but unlike Merge Sort, this algorithm does in place sorting.

Quick sort is a sorting algorithm based on the partition and exchange that picks elements as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct in the sorted array.
The key process in quick Sort is a partition. The target of partitions is to place the pivot (any element can be chosen to be a pivot) at its correct position in the sorted array and put all smaller elements to the left of the pivot, and all greater elements to the right of the pivot.
Partition is done recursively on each side of the pivot after the pivot is placed in its correct position and this finally sorts the array.

Procedure:

- Open any C editor
- Write a C program
- Compile and run the code
- Note down the Output

Quicksort is a divide and conquer algorithm.

It divides the large array into smaller sub-arrays. And then quicksort recursively sort the sub-arrays.e

Steps of QuickSort Algorithm:

1.Choose a Pivot: Select an element from the array as the pivot. Common strategies include:

1. First element
2. Last element
3. Random element
4. Median of three

2.Partitioning:

1. Rearrange the elements so that:
    1. Elements smaller than the pivot are placed to its left.
    2. Elements greater than the pivot are placed to its right.
2. The pivot is now in its correct position.

Recursively Apply QuickSort:

1. Apply QuickSort on the left subarray (elements less than the pivot).
2. Apply QuickSort on the right subarray (elements greater than the pivot).

Base Case:

1. When the subarray has one or zero elements, stop recursion.

Given Array:

`[10, 15, 1, 2, 9, 16, 11]`
Pivot Element: First element (`10`)

Step 1: Partitioning around Pivot (10)
[10, 15, 1, 2, 9, 16, 11]
↑
Pivot

- Elements less than 10: [1, 2, 9]
- Elements greater than 10: [15, 16, 11]

[9, 2, 1, 10, 15, 16, 11]
↑
Pivot placed

Pivot index = 3

Step 2: Quick Sort Left Subarray [9, 2, 1]
[9, 2, 1]
↑
Pivot

- Elements less than 9: [2, 1]
- No elements greater than 9

[1, 2, 9]
↑
Pivot placed

Pivot index = 2

Sorting [1, 2]
[1, 2]
↑
Pivot

- [1] is already sorted
- [2] is already sorted

Step 3: Quick Sort Right Subarray [15, 16, 11]
[15, 16, 11]
↑
Pivot

- Elements less than 15: [11]
- Elements greater than 15: [16]

[11, 15, 16]
↑
Pivot placed

Pivot index = 5

Final Sorted Array
[1, 2, 9, 10, 11, 15, 16]

This completes the Quick Sort process! 🚀

Program:

```
#include <stdio.h>
void quicksort(int array[], int first, int last){
   int i, j, pivot, temp;
    if (first < last)
        {    pivot = first; // Choosing first element as pivot
       i = first;
       j = last;
       while (i < j) {
          while (array[i] <= array[pivot] && i < last)
             i++;
          while (array[j] > array[pivot])
             j--;
          if (i < j) { // Swap elements at i and j
             temp = array[i];
             array[i] = array[j];
             array[j] = temp;
          }
       }
       // Swap pivot with array[j] to place it in the correct position
       temp = array[pivot];
       array[pivot] = array[j];
       array[j] = temp;
       // Recursively sort the sub-arrays
       quicksort(array, first, j - 1);
       quicksort(array, j + 1, last);
   }
}
int main() { // Changed void main() to int main()
   int i, n, array[25];
   printf("How many elements are you going to enter? ");
   scanf("%d", &n);
   printf("Enter %d elements: ", n);
   for (i = 0; i < n; i++)
      scanf("%d", &array[i]);
   quicksort(array, 0, n - 1);
   printf("The sorted elements are:\n");
   for (i = 0; i < n; i++)
```

```
        printf("%d ", array[i]);
    return 0; // Return 0 to indicate successful execution
}
```

OUTPUT:

How many number of elements you are going to enter?: 9
Enter 9 elements: 7 6 10 5 9 2 1 15 7
The Sorted elements are:
1 2 5 6 7 7 9 10 15

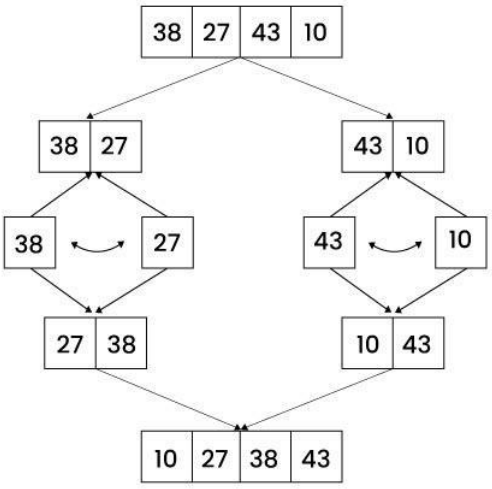
Merge Sort Algorithm:

Merge sort is a sorting algorithm that works by dividing an array into smaller sub arrays, sorting each sub array, and then merging the sorted sub arrays back together to form the final sorted array.
This process is repeated until the entire array is sorted.

Procedure:

Here is an example of how merge sort works:

Let's say we have an unsorted array of integers:



Merge Sort

- Finally, the array is completely sorted.
- Result of sorted array is 0 , 2 , 4 ,6.

Aspect	Complexity
Time Complexity	O(n log n)
Space Complexity	O(n)

Program:

```
#include <stdio.h>
// Function to merge two halves of an array
void merge(int arr[], int low, int mid, int high)
{
   int b[10]; // Temporary array
   int i = low, j = mid + 1, k = 0;
   // Merge elements in sorted order
   while (i <= mid && j <= high)
         {
      if (arr[i] < arr[j])
                  {
        b[k] = arr[i];
        i++;
       }
        else
                  {
        b[k] = arr[j];
        j++;        }
      k++;
   }
   // Copy remaining elements from left subarray
   while (i <= mid)
         {
      b[k] = arr[i];
      i++;       k++;
   }
        // Copy remaining elements from right subarray
   while (j <= high)
          {
      b[k] = arr[j];
      j++;       k++;
   }
```

```
    // Copy merged elements back to original array

for (i = low, k = 0; i <= high; i++, k++)
        arr[i] = b[k];
}
// Function to perform merge sort
void mergeSort(int arr[], int left, int right)
 {
    if (left < right)
          {
        int mid = (left + right) / 2; // Find the middle point
        // Recursively sort first and second halves
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        // Merge the sorted halves
        merge(arr, left, mid, right);
    }
}
// Main function
int main()
{
    int a[25],i,n;
    printf("How many elements are you going to enter? ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for(i=0;i<n;i++)
         scanf("%d",&a[i]);
    mergeSort(a, 0, n - 1);
    printf("Sorted array:\n");
         for(i=0;i<n;i++)
                printf("%d\t",a[i]);
    return 0;
}
```

OUTPUT:

```
Enter the number of elements to sort:6
Enter the elements:
40 60 30 50 10 20
 List after sorting is
10 20 30 40 50 60
```

Radix Sort Algorithm:

Radix Sort is a **non-comparative** sorting algorithm that sorts numbers by processing individual digits. It works by sorting numbers **digit by digit**, starting from the least significant digit (LSD) to the most significant digit (MSD) using a **stable sorting algorithm** (like counting sort).

1. **Find the Maximum Number**: Determine the largest number in the array to know the number of digits.
2. **Sort Digit by Digit**:
    1. Sort the numbers based on the least significant digit (LSD).
    2. Move to the next significant digit and repeat.
    3. Continue until the most significant digit (MSD) is sorted.

The Given array is

0	1	2	3	4	5	6	7	8	9
432	8	530	90	88	231	11	45	677	199

Largest=677
Nod=3//It will take 3 iterations
Iteration 1: Consider units position

- Insert 432 at b[2][0] and increments count[2] by 1
- Insert 8 at b[8][0] and increment count[8] by 1
- Insert 530 at b[0][0] and increment count[0] by 1
- Insert 90 at b[0][1] and increment count[0] by 1
- Insert 88 at b[8][1] and increment count[8] by 1
- Insert 231 at b[1][0] and increment count[1] by 1
- Insert 11 at b[1][1] and increment count[1] by 1
- Insert 45 at b[5][0] and increment count[5] by 1
- Insert 677 at b[7][0] and increment count[7] by 1
- Insert 199 at b[9][0] and increment count[9] by 1

	Count	b	0	1	2	3	4	5	6	7	8	9
0	2	0	530	90								
1	2	1	231	11								
2	1	2	432									
3	0	3										
4	0	4										
5	1	5	45									
6	0	6										
7	1	7	677									
8	2	8	8	88								
9	1	9	199									

- Arrange the elements in above bucket from top to bottom

0	1	2	3	4	5	6	7	8	9
530	90	231	11	432	45	677	8	88	199

Iteration 2: Consider tens position

	Count	b	0	1	2	3	4	5	6	7	8	9
0	1	0	8									
1	1	1	11									
2	0	2										
3	3	3	530	231	432							
4	1	4	45									
5	0	5										
6	0	6										
7	1	7	677									
8	1	8	88									
9	2	9	90	199								

- Insert 530 at b[3][0] and increments count[3] by 1
- Insert 90 at b[9][0] and increment count[9] by 1
- Insert 231 at b[3][1] and increment count[3] by 1
- Insert 11 at b[1][0] and increment count[1] by 1
- Insert 432 at b[3][2] and increment count[3] by 1
- Insert 45 at b[4][0] and increment count[4] by 1
- Insert 677 at b[7][0] and increment count[7] by 1

- Insert 8 at b[0][0] and increment count[0] by 1
- Insert 88 at b[8][0] and increment count[8] by 1
- Insert 199 at b[9][1] and increment count[9] by 1

- Arrange the elements in above bucket from top to bottom

0	1	2	3	4	5	6	7	8	9
8	11	530	231	432	45	677	88	90	199

Iteration 3: Consider hundreds position

- Insert 8 at b[0][0] and increments count[0] by 1
- Insert 11 at b[0][1] and increment count[0] by 1
- Insert 530 at b[5][0] and increment count[5] by 1
- Insert 231 at b[2][0] and increment count[2] by 1
- Insert 432 at b[4][0] and increment count[4] by 1
- Insert 45 at b[0][2] and increment count[0] by 1
- Insert 677 at b[6][0] and increment count[6] by 1
- Insert 88 at b[0][3] and increment count[0] by 1
- Insert 90 at b[0][4] and increment count[0] by 1
- Insert 199 at b[1][0] and increment count[1] by 1

	Count	b	0	1	2	3	4	5	6	7	8	9
0	5	0	8	11	45	88	90					
1	1	1	199									
2	1	2	231									
3	0	3										
4	1	4	432									
5	1	5	530									
6	1	6	677									
7	0	7										
8	0	8										
9	0	9										

- Arrange the elements in above bucket from top to bottom

0	1	2	3	4	5	6	7	8	9
8	11	45	88	90	199	231	432	530	677

Program:

```
#include <stdio.h>
// Function to get the maximum value in an array
int getMax(int a[], int n)
{
  int max = a[0], i;
  for(i = 1; i < n; i++)
 {
    if(a[i] > max)
       max = a[i];
  }
  return max; // Maximum element from the array
}
// Function to implement radix sort
void radixSort(int a[], int n)
 {
        int big, nod = 0, steps, count[10];
        int i, j, k, bucket[10][n], loc, div = 1;
        big = getMax(a, n);
  // Count the number of digits in the largest number
        while (big > 0)
        {
                nod++;
                big = big / 10;
        }


for (steps = 1; steps <= nod; steps++)
{
   // Initialize count array
   for (j = 0; j < 10; j++)
         {    count[j] = 0;    }
    // Distribute elements into buckets
   for (i = 0; i < n; i++)
         {
                loc = (a[i] / div) % 10;
                bucket[loc][count[loc]++] = a[i];
    }
   // Collect elements back into the array
   k = 0;
```

```
    for (j = 0; j < 10; j++)
          {  // Looping through digits 0-9
          for (i = 0; i < count[j]; i++)
            {
                  a[k++] = bucket[j][i];
            }
          }
     // Move to the next digit
     div = div * 10;
   }
 }

// Function to print array elements
void printArray(int a[], int n)
 {
  int i;
  for (i = 0; i < n; ++i)
{
    printf("%d  ", a[i]);
  }
  printf("\n");
}

int main()
{
  int a[100],n,i;
  printf("\nEnter No.of elements ");
  scanf("%d",&n);
  for (i=0;i<n;i++)
  {
        scanf("%d",&a[i]);
  }
  printf("Before sorting, array elements are:\n");
  printArray(a, n);
  radixSort(a, n);
  printf("After applying Radix sort, array elements are:\n");
  printArray(a, n);
  return 0;
}
```

Output:

enter the range:5
Enter the elements:
5 4 1 3 2
Array after sorting: 1 2 3 4 5